

Robot Learning: From Fundamentals to Foundation Models

Lecture 7: Sequence Modeling & Transformers

Oier Mees

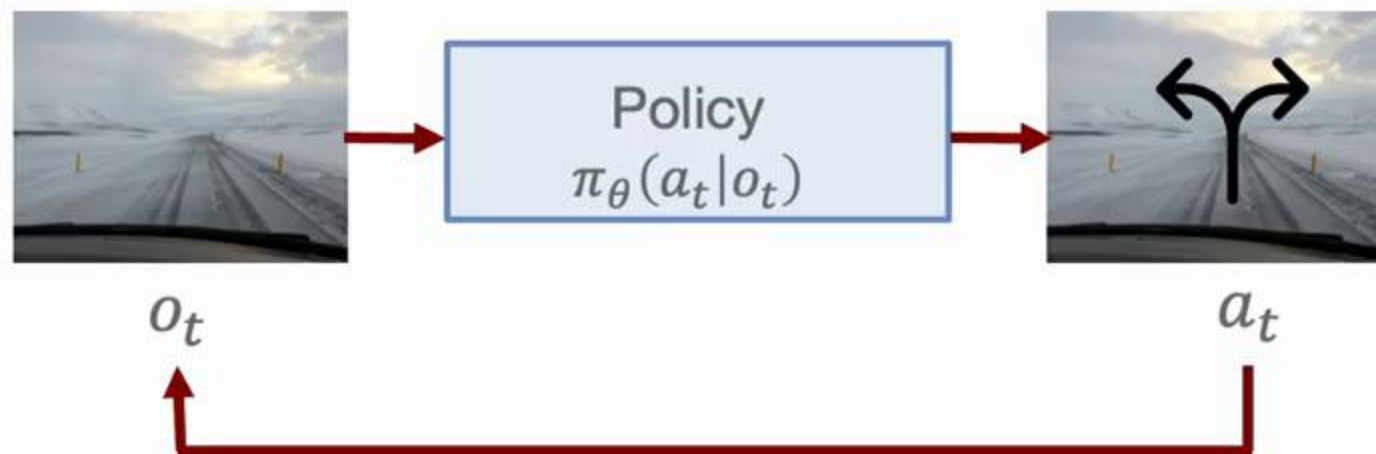
ETH zürich



Microsoft

30.03.2026

Recap: Limitations of Reactive Policies



One Snapshot Can't Model Full State

Memory: what did I do 10 seconds ago? How fast are pedestrians moving?

Action Smoothness: single step model might produce jerky movements

Robotics as a Sequence Modeling Problem

- Robots usually operate in a POMDP, which require reasoning over history $\tau = (o_0, a_0, o_1, a_1, \dots, o_T)$



$$p_{\pi}(\tau) = \rho(s_0) \prod_{t=0}^{T-1} \pi(a_t | o_t) P(o_{t+1} | o_t, a_t)$$



Trajectory is a sequence, but the policy is single-step and reactive

Autoregressive Models

- Any joint distribution can be written as a product of conditionals (chain rule of probability)

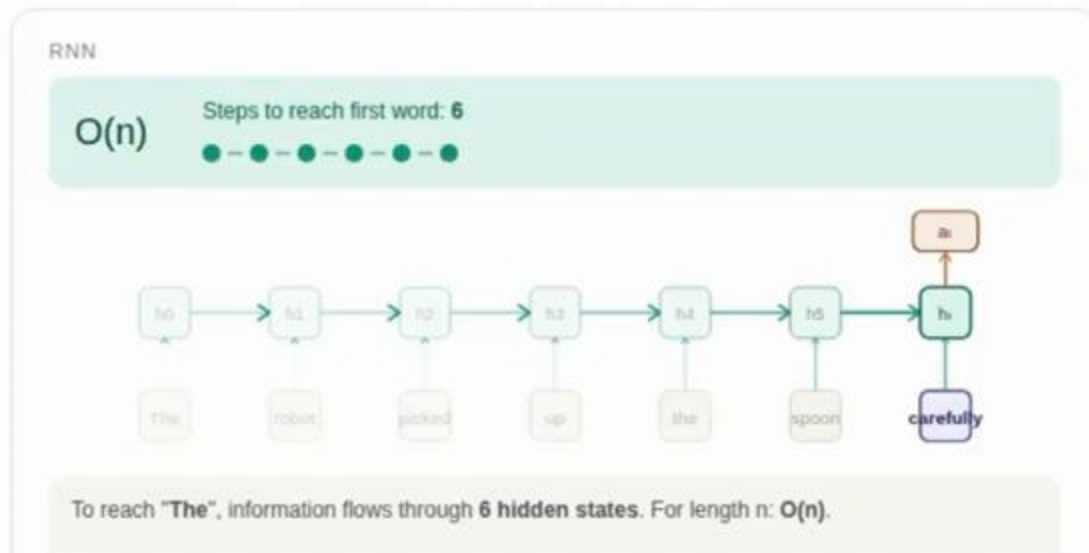
$$p_{\theta}(x) = \prod_{i=1}^T p_{\theta}(x_i \mid x_{1:i-1})$$



How do we learn each conditional?

Modeling Sequences with RNNs

- SOTA until 2016 in NLP, process sequences sequentially
- Limitations:
 - Learn long-range dependencies, exploding/vanishing gradients
 - Parallelize training due to recurrence
 - Retain information, fixed-size h_t state compresses all history



$$h_t = f(h_{t-1}, x_t)$$

Long Short-Term Memory

Hochreiter & Schmidhuber (1997)

Sequence to Sequence Learning with Neural Networks

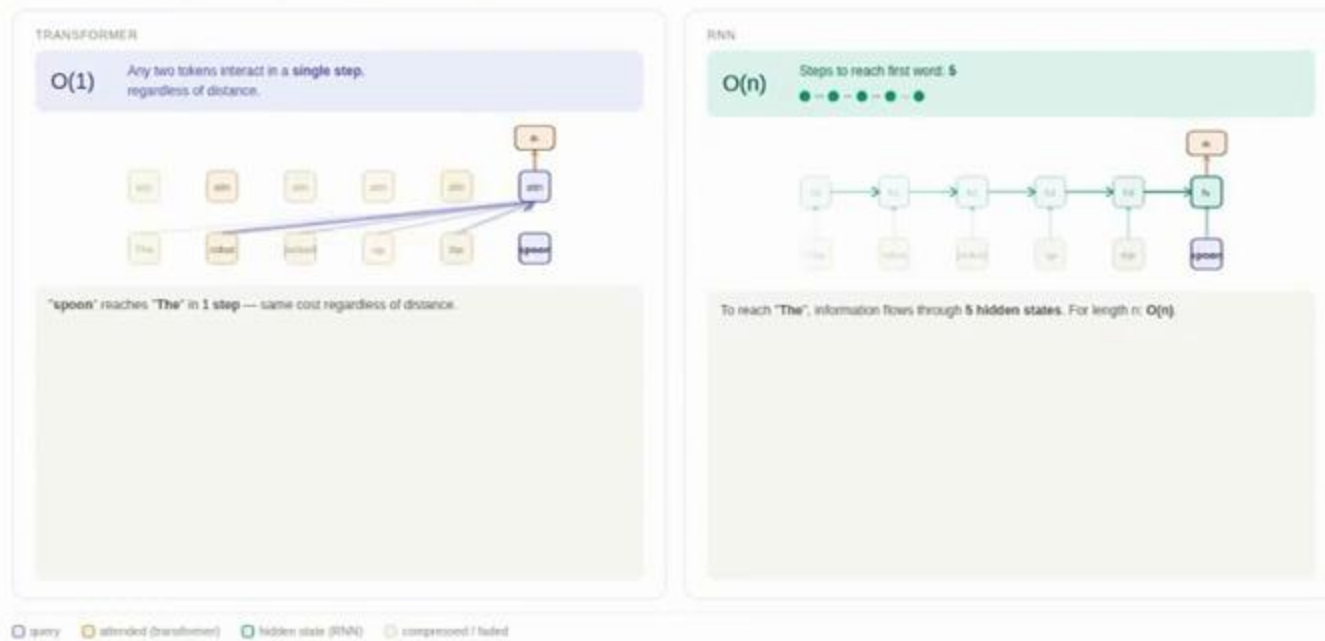
Sutskever et al., (2014)

Transformer

- Replace recurrence with attention, giving direct $O(1)$, equal access to the entire history

Transformer vs RNN: path length to attend past words

The robot picked up the spoon carefully



Attention Is All You Need
Vaswani et al., (2017)

Transformer: Attention Mechanism

Step 1 — embed 2 — project Q,K,V 3 — raw scores 4 — softmax 5 — weighted sum

The robot picked up the spoon

$$Q = eW_Q, K = eW_K, V = eW_V$$

Every token creates three vectors via learned weight matrices: Query (what am I looking for?), Key (what do I advertise to others?), Value (what I will contribute if attended to).

Query Key Value Output



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

Step 1 — embed 2 — project Q,K,V 3 — raw scores 4 — softmax 5 — weighted sum

The robot picked up the spoon



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\text{score}(q, k_i) = q \cdot k_i / \sqrt{d}$$

The query vector for "spoon" is dot-producted with every Key vector. A high score means that key matches what the query is looking for. These are raw unnormalized scores.

Raw dot-product scores (unnormalized)



unnormalized — does not sum to 1

□ Query □ Key □ Value □ Output

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

Step 1 — embed 2 — project Q,K,V 3 — raw scores 4 — softmax 5 — weighted sum

The robot picked up the spoon



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\alpha_i = \exp(\text{score}_i) / \sum_j \exp(\text{score}_j)$$

Softmax normalizes the raw scores into probabilities summing to 1. High scores become dominant weights; low scores are suppressed. These are the final attention probabilities α_i .

Softmax weights α (sum = 1)



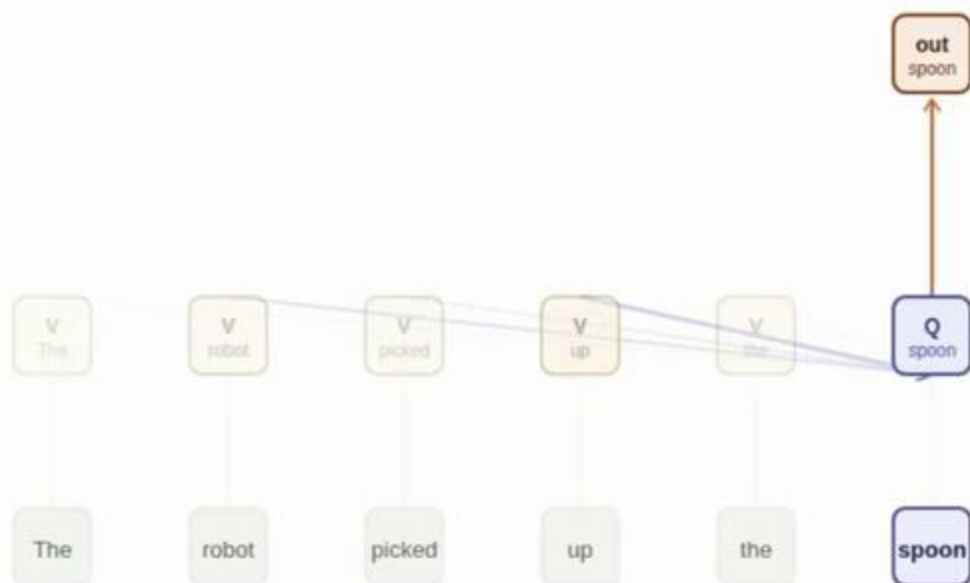
Query Key Value Output

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

Step 1 — embed 2 — project Q,K,V 3 — raw scores 4 — softmax 5 — weighted sum

The robot picked up the spoon



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\text{output} = \sum_i \alpha_i \cdot v_i$$

The output is a weighted blend of all Value vectors, weighted by α_i . "spoon" now carries a blended representation absorbing context from the entire sequence it attended to.

Output = weighted blend of value vectors

The	2.6%	■	$v(\text{The})$
robot	15.8%	■	$v(\text{robot})$
picked	9.6%	■	$v(\text{picked})$
up	28.8%	■	$v(\text{up})$
the	4.3%	■	$v(\text{the})$
spoon	38.9%	■	$v(\text{spoon})$

output("spoon")

$$= \sum_i \alpha_i \cdot v_i$$

Query Key Value Output

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

Step 1 — embed 2 — project Q,K,V 3 — raw scores 4 — softmax 5 — weighted sum

The robot picked up the spoon

$$Q = eW_Q, K = eW_K, V = eW_V$$

Every token creates three vectors via learned weight matrices: Query (what am I looking for?), Key (what do I advertise to others?), Value (what I will contribute if attended to).

Query Key Value Output



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

Step 1 — embed 2 — project Q,K,V 3 — raw scores 4 — softmax 5 — weighted sum

The robot picked up the spoon



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\text{score}(q, k_i) = q \cdot k_i / \sqrt{d}$$

The query vector for "spoon" is dot-producted with every Key vector. A high score means that key matches what the query is looking for. These are raw unnormalized scores.

Raw dot-product scores (unnormalized)



unnormalized — does not sum to 1

□ Query □ Key □ Value □ Output

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

Step 1 — embed 2 — project Q,K,V 3 — raw scores 4 — softmax 5 — weighted sum

The robot picked up the spoon

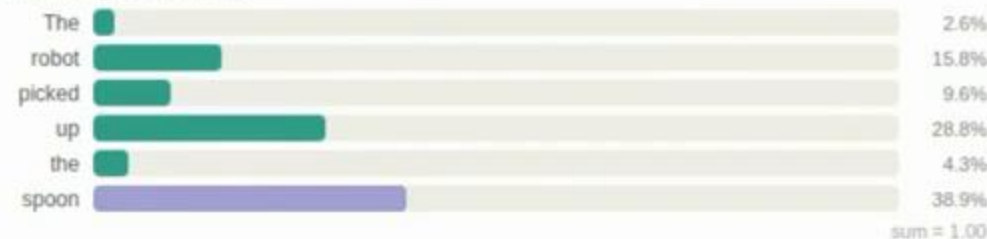


$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\alpha_i = \exp(\text{score}_i) / \sum_j \exp(\text{score}_j)$$

Softmax normalizes the raw scores into probabilities summing to 1. High scores become dominant weights; low scores are suppressed. These are the final attention probabilities α_i .

Softmax weights α_i (sum = 1)



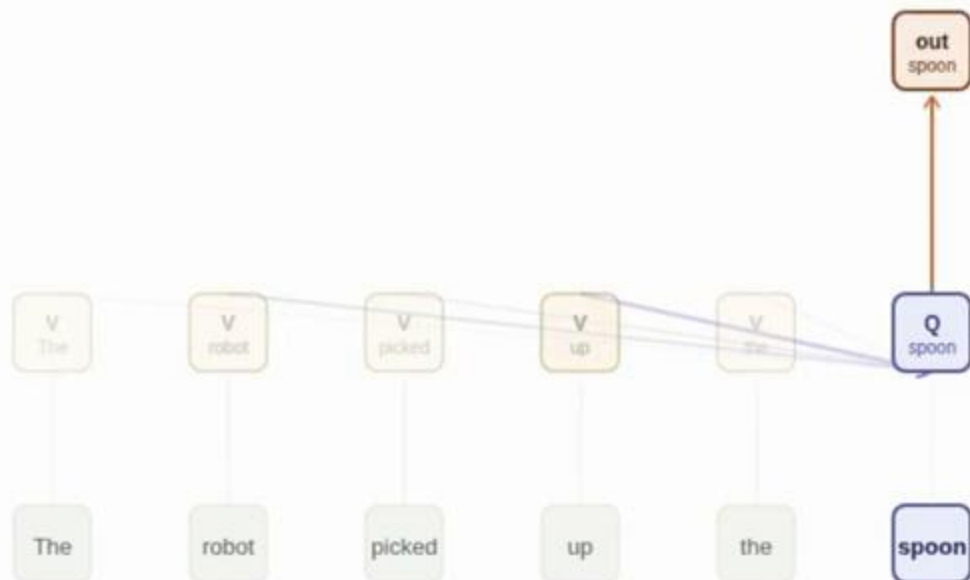
Query Key Value Output

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

Step 1 — embed 2 — project Q,K,V 3 — raw scores 4 — softmax 5 — weighted sum

The robot picked up the spoon



$$\text{output} = \sum_i \alpha_i \cdot v_i$$

The output is a weighted blend of all Value vectors, weighted by α . "spoon" now carries a blended representation absorbing context from the entire sequence it attended to.

Output = weighted blend of value vectors

The	2.6%	$v(\text{The})$
robot	15.8%	$v(\text{robot})$
picked	9.6%	$v(\text{picked})$
up	28.8%	$v(\text{up})$
the	4.3%	$v(\text{the})$
spoon	38.9%	$v(\text{spoon})$

output("spoon")

$$= \sum_i \alpha_i \cdot v_i$$

Query Key Value Output

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

Step 1 — embed 2 — project Q,K,V 3 — raw scores 4 — softmax 5 — weighted sum

The robot picked up the spoon

$$Q = eW_Q, K = eW_K, V = eW_V$$

Every token creates three vectors via learned weight matrices: Query (what am I looking for?), Key (what do I advertise to others?), Value (what I will contribute if attended to).

□ Query □ Key □ Value □ Output



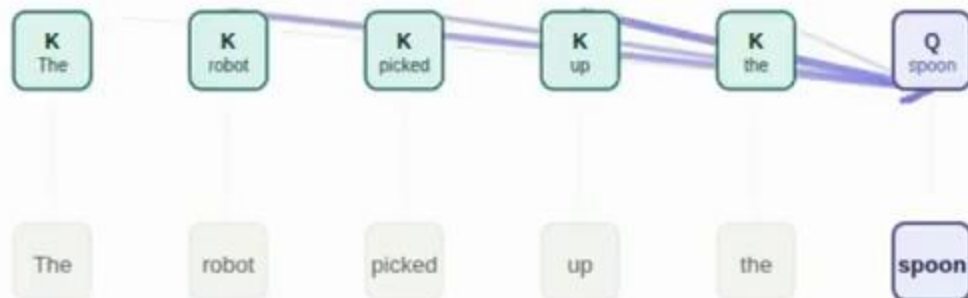
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

Step 1 — embed 2 — project Q,K,V 3 — raw scores 4 — softmax 5 — weighted sum

The robot picked up the spoon



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\text{score}(q, k_i) = q \cdot k_i / \sqrt{d}$$

The query vector for "spoon" is dot-producted with every Key vector. A high score means that key matches what the query is looking for. These are raw unnormalized scores.

Raw dot-product scores (unnormalized)



unnormalized — does not sum to 1

Query Key Value Output

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

Step 1 — embed 2 — project Q,K,V 3 — raw scores 4 — softmax 5 — weighted sum

The robot picked up the spoon

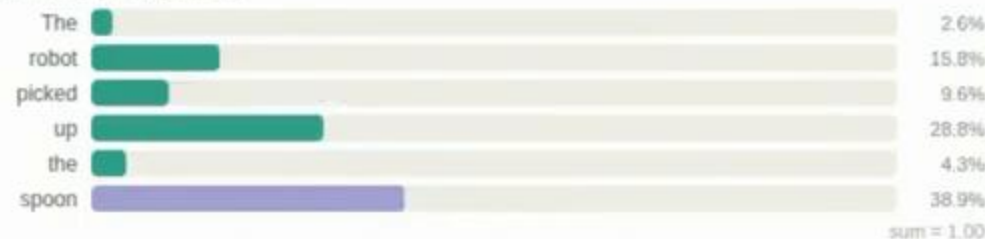


$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\alpha_i = \exp(\text{score}_i) / \sum_j \exp(\text{score}_j)$$

Softmax normalizes the raw scores into probabilities summing to 1. High scores become dominant weights; low scores are suppressed. These are the final attention probabilities α_i .

Softmax weights α_i (sum = 1)



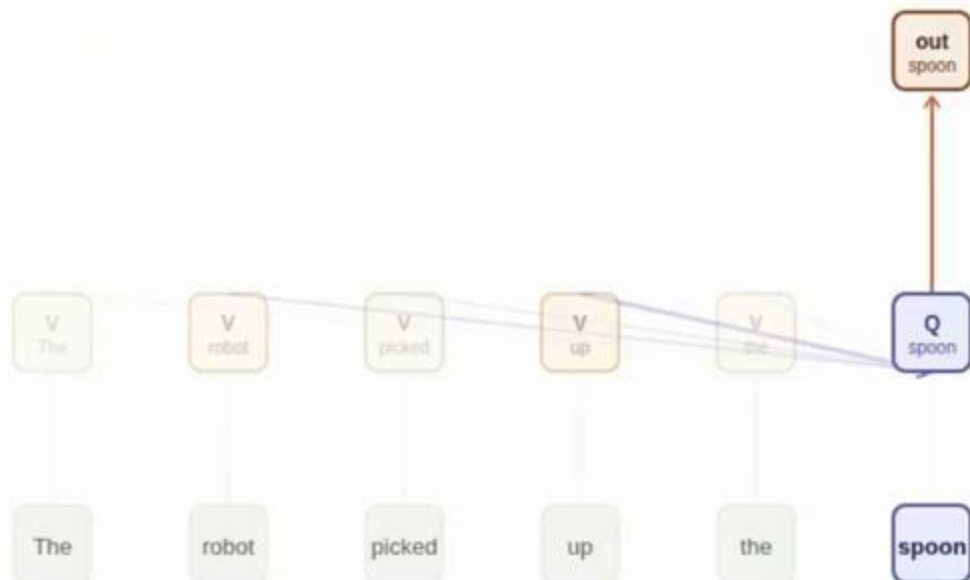
Query Key Value Output

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

Step 1 — embed 2 — project Q,K,V 3 — raw scores 4 — softmax 5 — weighted sum

The robot picked up the spoon



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\text{output} = \sum_i \alpha_i \cdot v_i$$

The output is a weighted blend of all Value vectors, weighted by α_i . "spoon" now carries a blended representation absorbing context from the entire sequence it attended to.

Output = weighted blend of value vectors

The	2.6%	■	$v(\text{The})$
robot	15.8%	■	$v(\text{robot})$
picked	9.6%	■	$v(\text{picked})$
up	28.8%	■	$v(\text{up})$
the	4.3%	■	$v(\text{the})$
spoon	38.9%	■	$v(\text{spoon})$

output("spoon")

$$= \sum_i \alpha_i \cdot v_i$$

Query Key Value Output

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

Step 1 — embed 2 — project Q,K,V 3 — raw scores 4 — softmax 5 — weighted sum

The robot picked up the spoon

$$Q = eW_Q, K = eW_K, V = eW_V$$

Every token creates three vectors via learned weight matrices: Query (what am I looking for?), Key (what do I advertise to others?), Value (what I will contribute if attended to).

Query Key Value Output



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

Step 1 — embed 2 — project Q,K,V 3 — raw scores 4 — softmax 5 — weighted sum

The robot picked up the spoon



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\text{score}(q, k_i) = q \cdot k_i / \sqrt{d}$$

The query vector for "spoon" is dot-producted with every Key vector. A high score means that key matches what the query is looking for. These are raw unnormalized scores.

Raw dot-product scores (unnormalized)



unnormalized — does not sum to 1

□ Query □ Key □ Value □ Output

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

Step 1 — embed 2 — project Q,K,V 3 — raw scores 4 — softmax 5 — weighted sum

The robot picked up the spoon



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\alpha_i = \exp(\text{score}_i) / \sum_j \exp(\text{score}_j)$$

Softmax normalizes the raw scores into probabilities summing to 1. High scores become dominant weights; low scores are suppressed. These are the final attention probabilities α_i .

Softmax weights α_i (sum = 1)



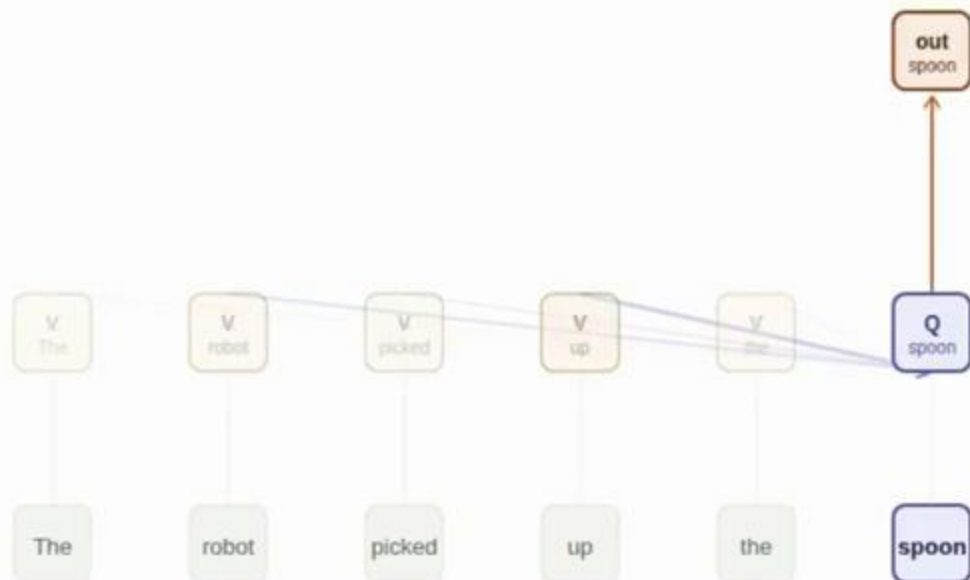
□ Query □ Key □ Value □ Output

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

Step 1 — embed 2 — project Q,K,V 3 — raw scores 4 — softmax 5 — weighted sum

The robot picked up the spoon



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\text{output} = \sum_i \alpha_i \cdot v_i$$

The output is a weighted blend of all Value vectors, weighted by α . "spoon" now carries a blended representation absorbing context from the entire sequence it attended to.

Output = weighted blend of value vectors

The	2.6%	■	$v(\text{The})$
robot	15.8%	■	$v(\text{robot})$
picked	9.6%	■	$v(\text{picked})$
up	28.9%	■	$v(\text{up})$
the	4.3%	■	$v(\text{the})$
spoon	38.9%	■	$v(\text{spoon})$

output("spoon")

$$= \sum_i \alpha_i \cdot v_i$$

Query Key Value Output

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

Step 1 — embed 2 — project Q,K,V 3 — raw scores 4 — softmax 5 — weighted sum

The robot picked up the spoon

$$Q = eW_Q, K = eW_K, V = eW_V$$

Every token creates three vectors via learned weight matrices: Query (what am I looking for?), Key (what do I advertise to others?), Value (what I will contribute if attended to).

Query Key Value Output



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

Step 1 — embed 2 — project Q,K,V 3 — raw scores 4 — softmax 5 — weighted sum

The robot picked up the spoon



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\text{score}(q, k_i) = q \cdot k_i / \sqrt{d}$$

The query vector for "spoon" is dot-producted with every Key vector. A high score means that key matches what the query is looking for. These are raw unnormalized scores.

Raw dot-product scores (unnormalized)



unnormalized — does not sum to 1

Query Key Value Output

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

Step 1 — embed 2 — project Q,K,V 3 — raw scores 4 — softmax 5 — weighted sum

The robot picked up the spoon



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\alpha_i = \exp(\text{score}_i) / \sum_j \exp(\text{score}_j)$$

Softmax normalizes the raw scores into probabilities summing to 1. High scores become dominant weights; low scores are suppressed. These are the final attention probabilities α_i .

Softmax weights α (sum = 1)



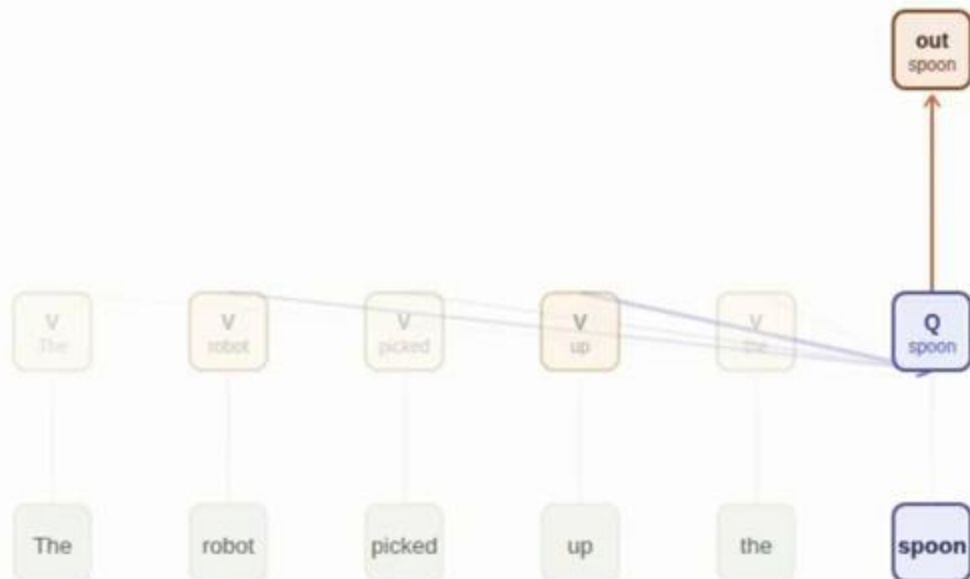
Query Key Value Output

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

Step 1 — embed 2 — project Q,K,V 3 — raw scores 4 — softmax 5 — weighted sum

The robot picked up the spoon



$$\text{output} = \sum_i \alpha_i \cdot v_i$$

The output is a weighted blend of all Value vectors, weighted by α_i . "spoon" now carries a blended representation absorbing context from the entire sequence it attended to.

Output = weighted blend of value vectors

The	2.6%	$\times$	$v(\text{The})$
robot	15.8%	$\times$	$v(\text{robot})$
picked	9.6%	$\times$	$v(\text{picked})$
up	28.8%	$\times$	$v(\text{up})$
the	4.3%	$\times$	$v(\text{the})$
spoon	38.9%	$\times$	$v(\text{spoon})$

output("spoon")

$$= \sum_i \alpha_i \cdot v_i$$

Query Key Value Output

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Positional Encodings

- Attention has no inherent notion of order
- Inject order information to tokens

Absolute Positional Encoding

A fixed vector PE is added to each token embedding before the transformer sees it.

① token embeddings



+

② fixed position vectors



=

③ inputs to transformer



Relative Positional Encoding

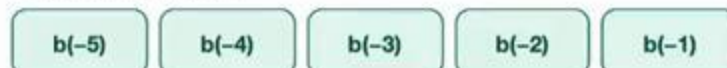
A bias $b(i-j)$ based on the distance between tokens is added to the attention score, not the embedding.

① context tokens (keys), query = "spoon"



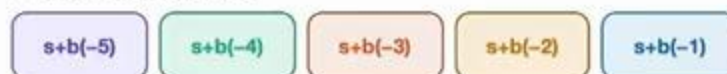
↓ offset from query

② relative bias per token



↓ added to attention score

③ biased attention scores



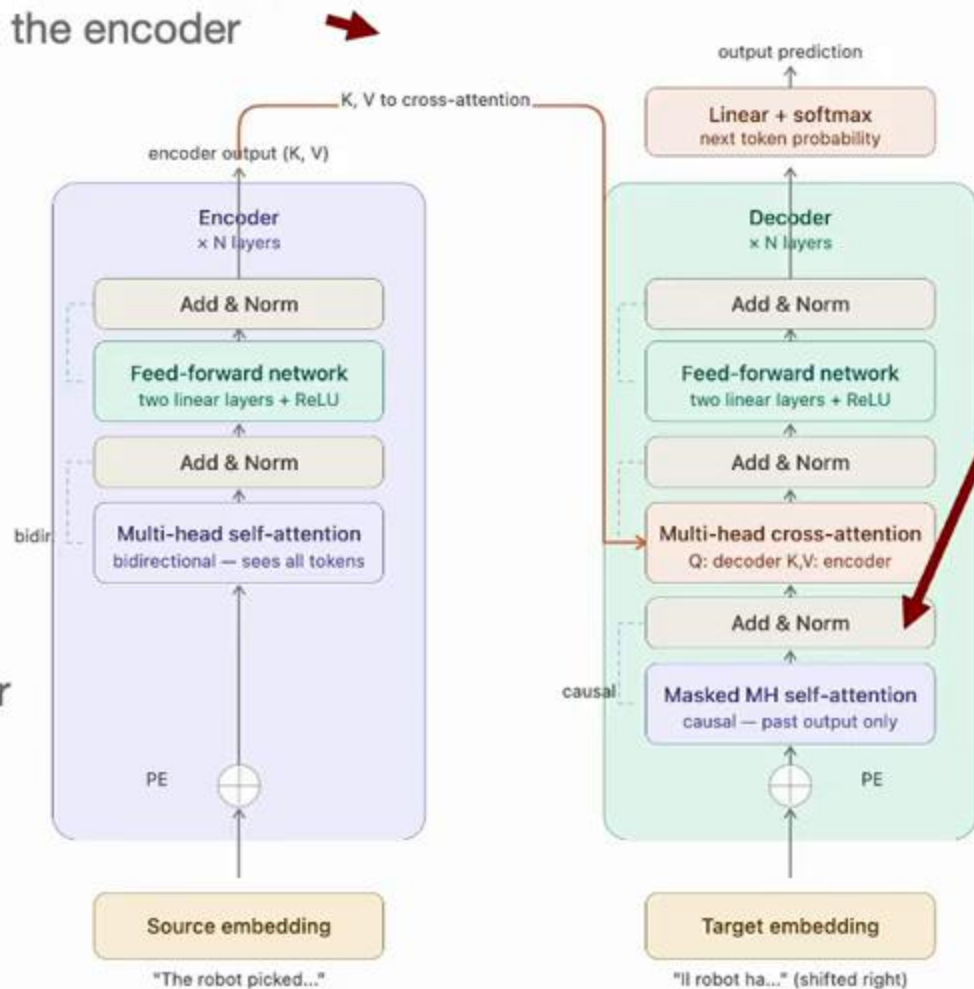
Original Transformer Architecture

Cross-attention

Same as self-attention, but Q comes from the decoder, K and V come from the encoder



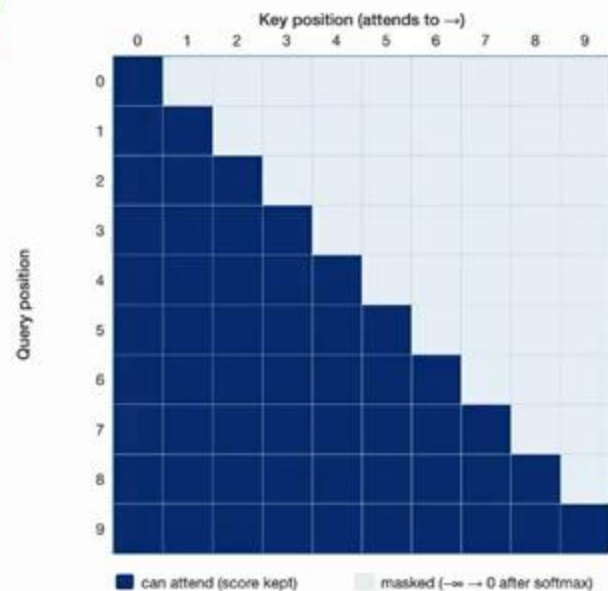
Encoder
Bidirectional self-attention
Every token attends to every other token, past and future



Decoder

Causal self-attention

Each token only attends to itself and past tokens



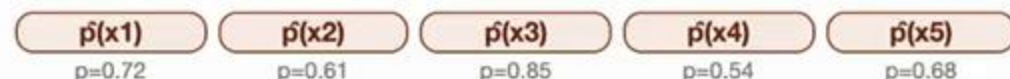
Training

input (ground truth, shifted right)

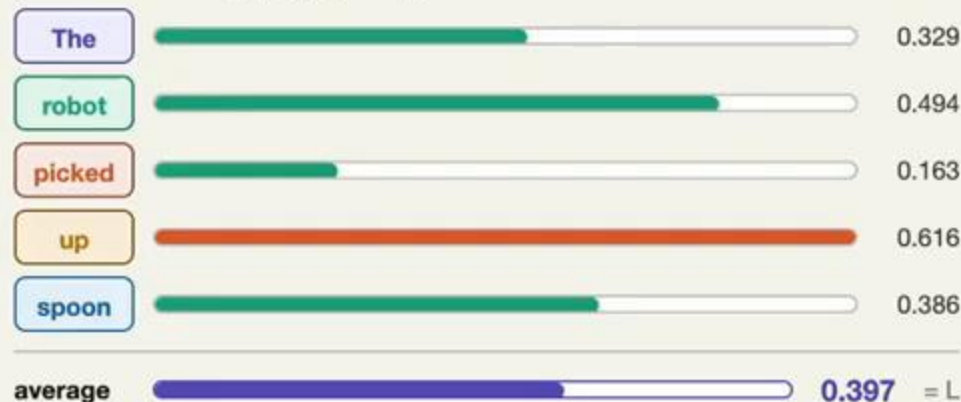


Decoder-only transformer + causal mask

predicted distribution (one step ahead)



per-token loss: $-\log p_{\theta}(x_t | x_{1..x_{t-1}})$



target (ground truth)



Teacher Forcing

maximize the likelihood of the next correct token x_t given the true preceding tokens $x_{1:t-1}$

Cross-Entropy Loss

$$L = -\frac{1}{T} \sum_{t=1}^T \log p_{\theta}(x_t | x_{<t})$$

- ✓ 1x forward pass computes all T losses with Transformers
- ✗ T forward passes with RNNs

Language Tokenization

- **Characters:** a=0, b=1, c=2, ...
 - Small vocabulary
 - Large number of tokens
- **Words:** cat=0, car=1, dog=2, ...
 - Large vocabulary
 - Small number of tokens
 - What about new words?

Ideally we would like something in between

Byte-Pair Encoding

- Tokenize based on groupings of characters, prioritizing by frequency
- Generalizes to novel combinations of characters

function BPE (strings C , number of merges k)

```
 $V \leftarrow$  all unique characters in  $C$            # initial vocabulary is characters
for  $i = 1$  to  $k$  do                               # repeat  $k$  times
     $t_L, t_R \leftarrow$  most frequent pair of adjacent tokens in  $C$ 
     $t_{NEW} \leftarrow t_L + t_R$                  # concatenate into new token
     $V \leftarrow V + t_{NEW}$                        # add to vocabulary
    replace each  $(t_L, t_R)$  in  $C$  with  $t_{NEW}$     # update corpus
return vocab  $V$ 
```

BPE Example

- BPE compression factor scales with corpus size



BPE Example

- BPE compression factor scales with corpus size

input sentence
"The robots melted the robot's unbearably cheesy fondue"

0 — start 1 — "th" **2 — "the"** 3 — "ro" 4 — "rob" 5 — "robo" 6 — "robot" 7 — result

current tokenization

T h e _ r o b o t s _ m e l t e d _ t h e _ r o b o t ' s _ u n b e a r a b l y _ c h e e s y _ f o n d u e

merge: th + e → the (2× in corpus)

top pair frequencies

'r'+'o'	3x
'o'+'b'	2x
'_'+'the'	2x
'the'+'_'	2x
'_'+'r'	2x
'e'+'e'	1x

'th'+'e' → 'the' (2 occurrences). Common function words emerge as single tokens early — they appear most frequently in any corpus.

vocabulary learned

th the

token count

52 tokens -2 from start

BPE Example

- BPE compression factor scales with corpus size

input sentence
"The robots melted the robot's unbearably cheesy fondue"

0 — start 1 — "th" 2 — "the" 3 — "ro" 4 — "rob" 5 — "robo" 6 — "robot" 7 — result

current tokenization

T h e _ robot s _ m e l t e d _ the _ robot ' s _ u n b e a r a b l y _ c h e e s y _ f o n d u e

merge: robo + t → robot (2x in corpus)

top pair frequencies

'robot'+'s'	1x
'robot'+' '	1x
'_'+'robot'	2x
'_'+'m'	1x
'e'+'a'	1x
'e'+'e'	1x

vocabulary learned

th the ro rob robo robot

token count

44 tokens -10 from start

'robo'+' ' → 'robot'. The full word is now a single token. Both 'robots' and 'robot's' share this token — BPE naturally handles morphological variants.

BPE Example

- BPE compression factor scales with corpus size

input sentence

"The robots melted the robot's unbearably cheesy fondue"

0 — start 1 — "th" 2 — "the" 3 — "ro" 4 — "rob" 5 — "robo" 6 — "robot" 7 — result

current tokenization

T h e _ robot s _ m e l t e d _ the _ robot ' s _ un b ear
ably _ ch ee s y _ f on d ue

After many merges: common words become single tokens. Rarer words split into subwords — 'unbearably' → 'un'+b'+ear'+ably', 'fondue' → 'f'+on'+d'+ue'. Any unknown word remains representable.

vocabulary learned

th the ro rob robo robot un ear ably ch ee on
ue

token count

34 tokens -20 from start

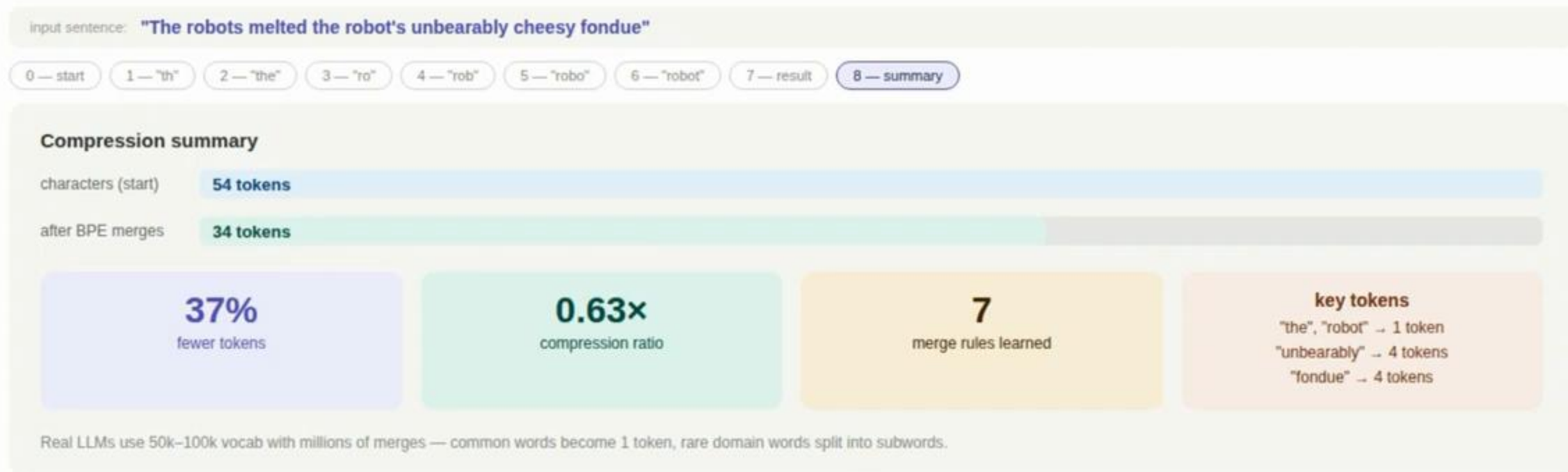
compression achieved

37% fewer tokens than characters

54 chars → 34 tokens compression ratio: 0.63×

BPE Example

- BPE compression factor scales with corpus size



Large Language Models

- Decoder-only architectures → simpler to scale
 - Extend BPE to work on byte-level → no unknown tokens
 - Techniques for scaling:
 - Rotary Position Embeddings (RoPE)
 - Rotates Q and K vectors by an angle proportional to their position, so that $Q \cdot K$ depends only on the relative distance between tokens
 - FlashAttention:
 - Attention with $O(n)$ memory instead of $O(n^2)$ by tiling the computation in fast SRAM, never materializing the full attention matrix
- RoPE + FlashAttention make training on longer sequences practical

Scaling LLMs

GPT-1
2018
117M params
1B tokens
Pretraining + fine-tuning

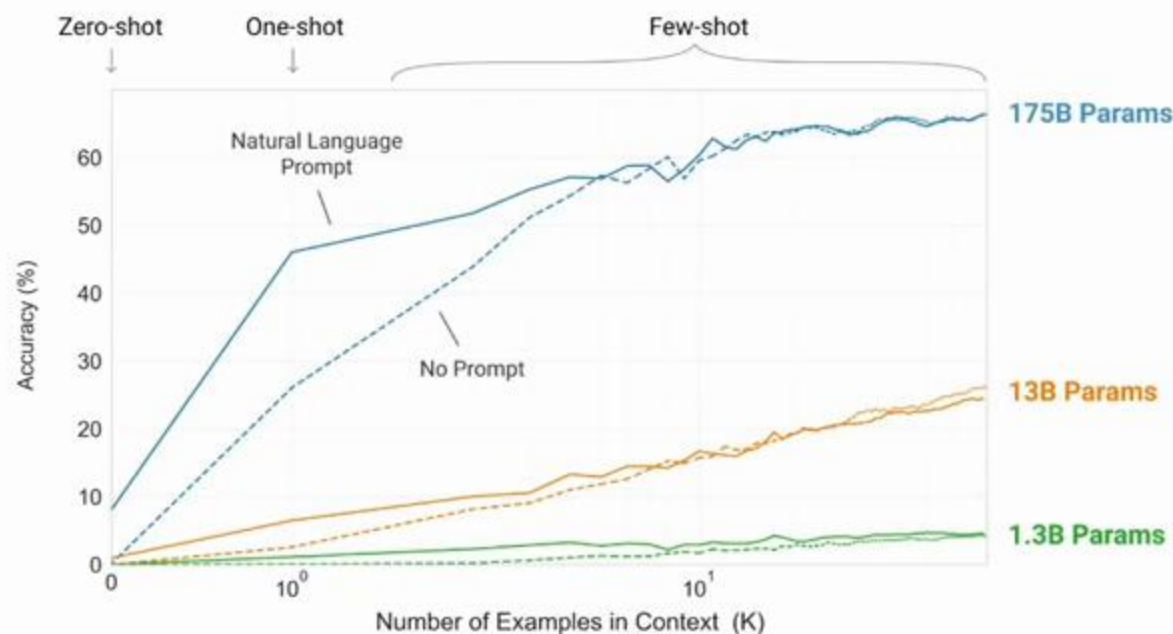
GPT-2
2019
1.5B params
10B tokens
Zero-shot generalization

GPT-3
2020
175B params
300B tokens
In-context learning emerges

The Bitter Lesson— Sutton (2019)



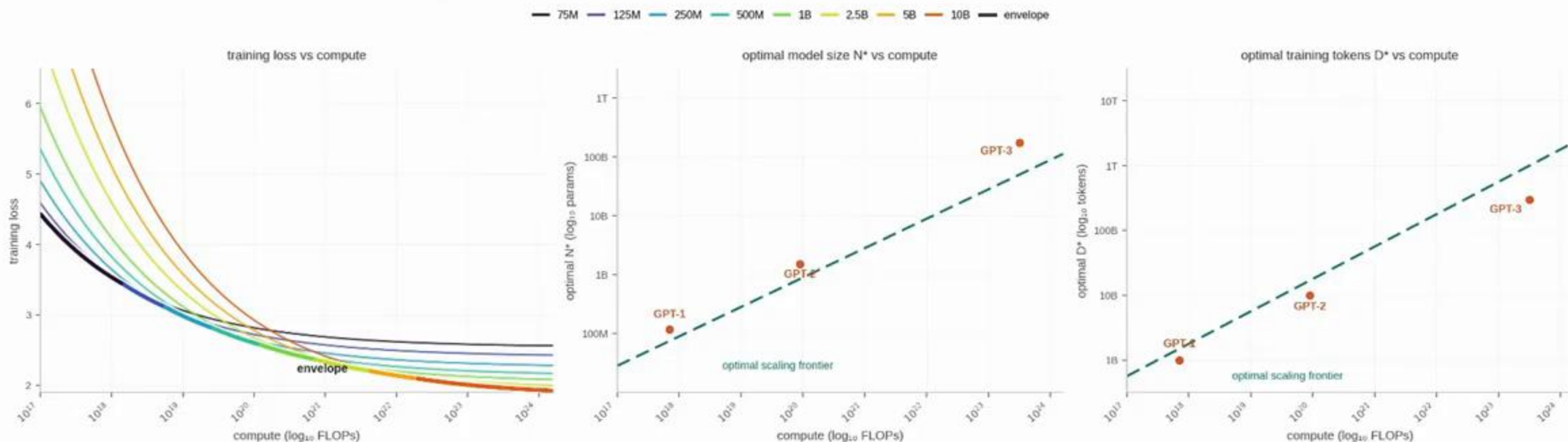
- Methods that scale with compute ultimately beat hand-engineered solutions
- ➔ In-context learning is an emergent property of scale



Language Models are Few-Shot Learners
Brown et al., OpenAI (2020)

Scaling Laws

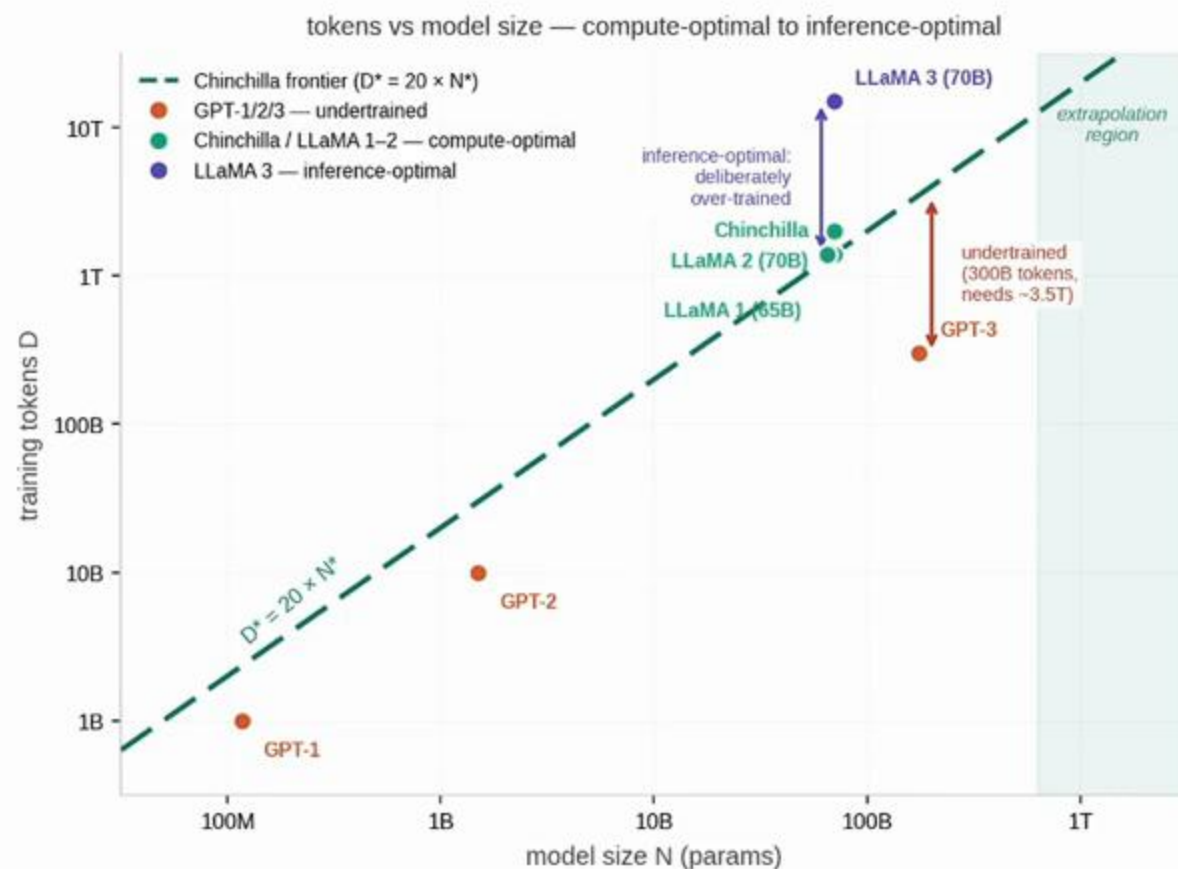
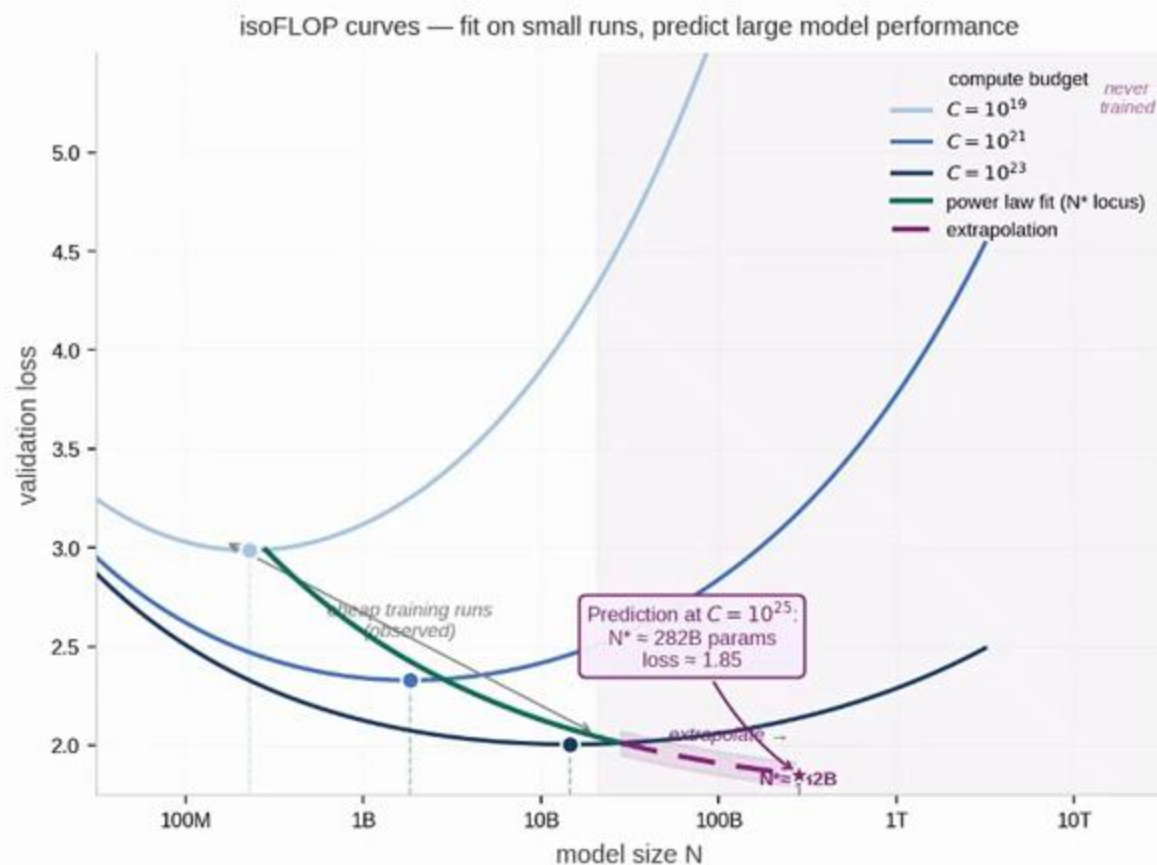
- Given compute budget, scale data & model size equally
 - Optimal model size $N^* \propto C^{0.5}$, data $D^* \propto C^{0.5}$, compute $C \approx 6 \times N \times D$
 - ~20 tokens per parameter for compute-optimality



Training Compute-Optimal Large Language Models
Hoffmann et al. (2022)

Scaling Laws: Extrapolating Performance

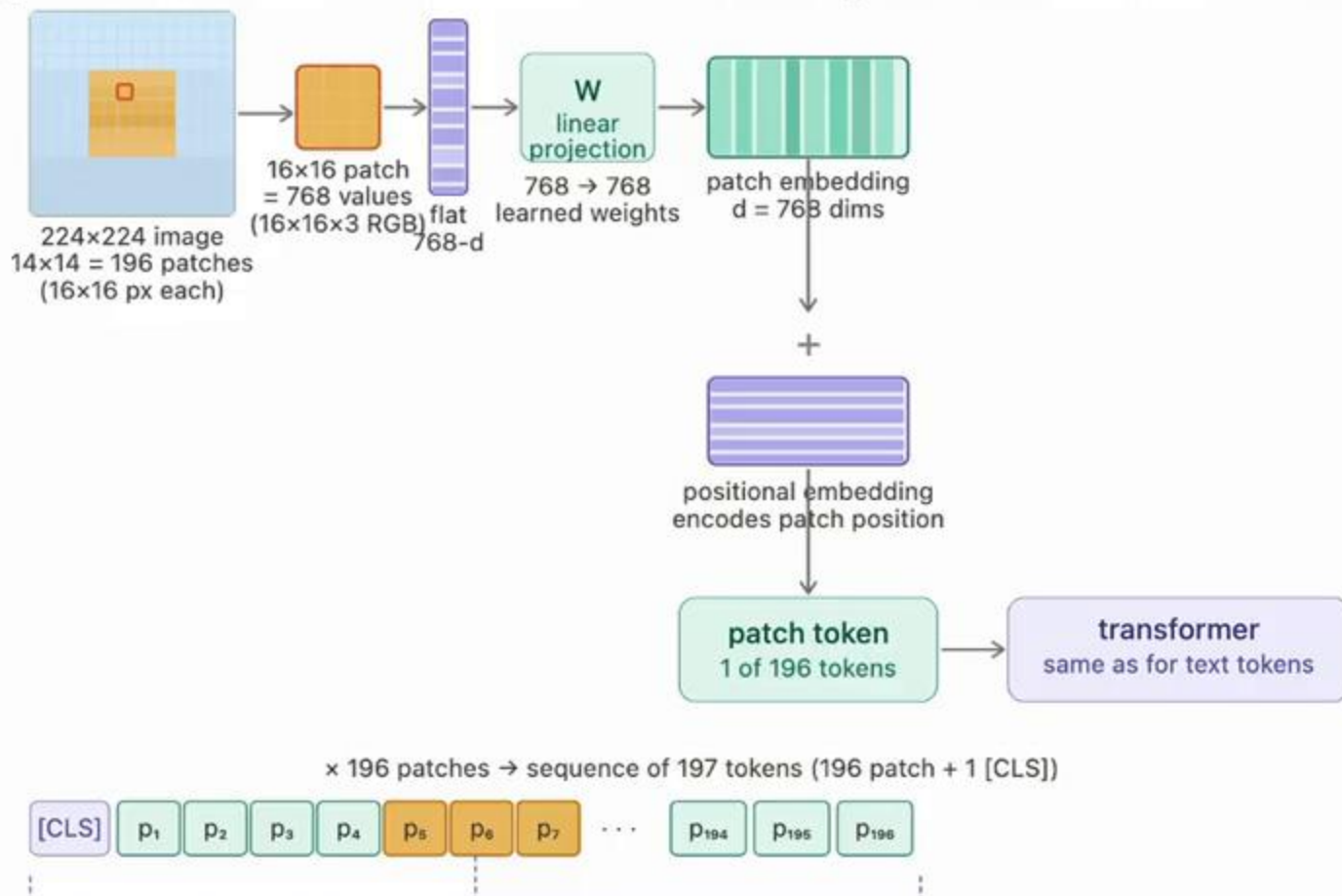
- Fit power law on cheap runs → extrapolate to predict optimal N and final loss before committing compute



How can we extend LLMs & Transformers to Vision?

Image Tokenization

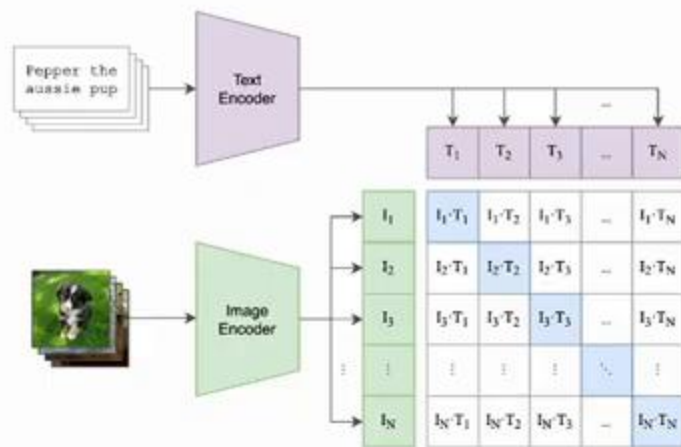
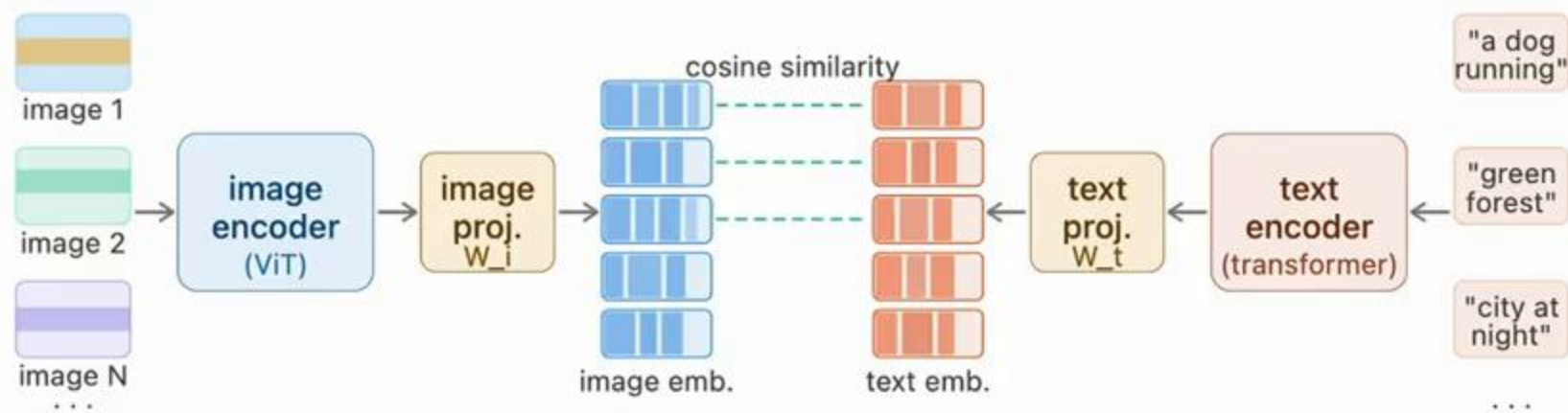
- Split image to patches, flatten & add positional encodings



An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale
Dosovitskiy, Beyer et al. (2020)

How does the Transformer know which text tokens correspond to which visual tokens?

CLIP: Two-Tower Contrastive Alignment



Learnable temperature, entropy of the resulting probability distribution

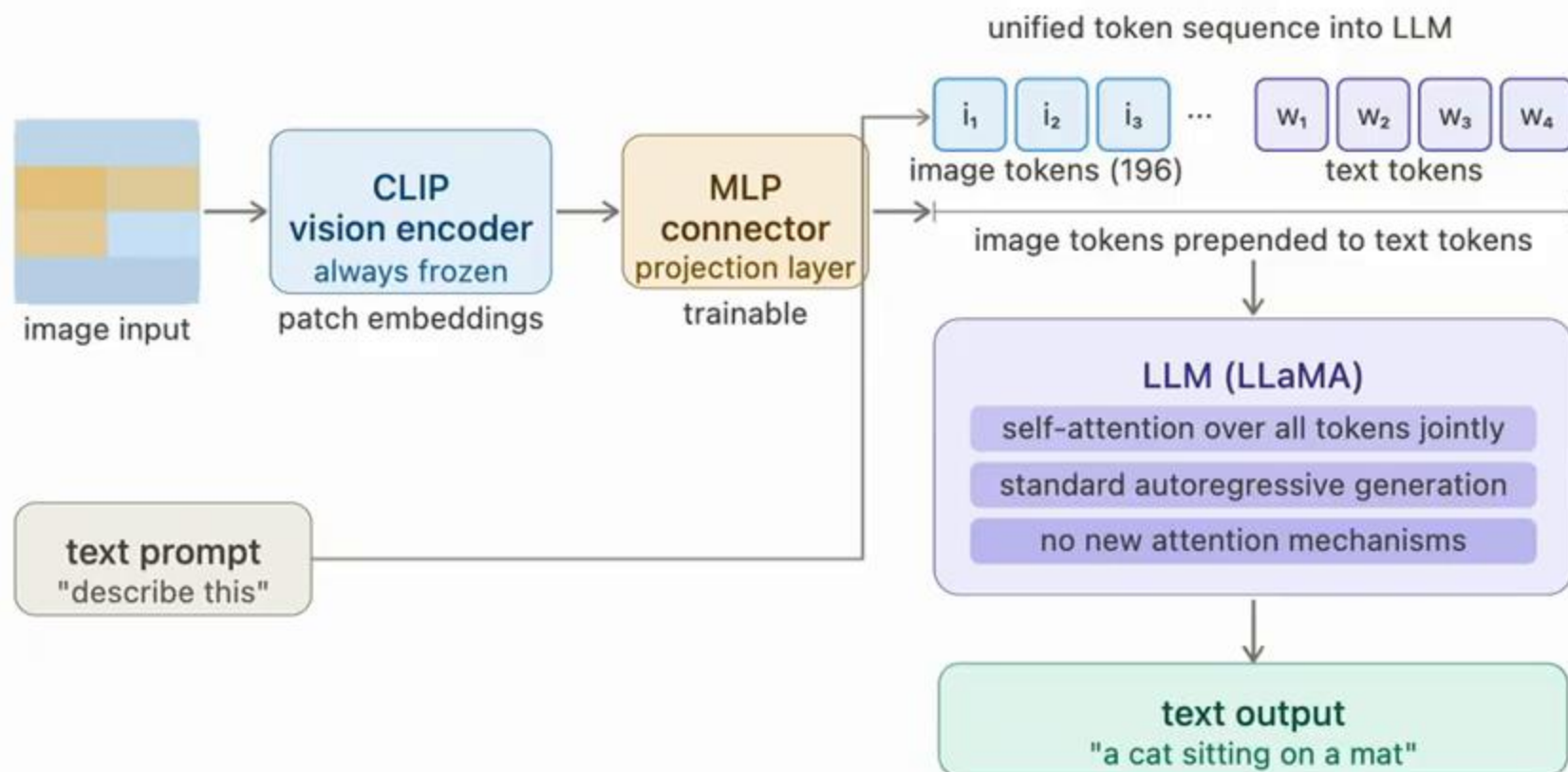
$$\mathcal{L}_{\text{CLIP}} = -\frac{1}{2N} \sum_{i=1}^N \left[\underbrace{\log \frac{e^{s_{ii}/\tau}}{\sum_{j=1}^N e^{s_{ij}/\tau}}}_{\text{Image-to-Text Loss}} + \log \frac{e^{s_{ii}/\tau}}{\sum_{j=1}^N e^{s_{ji}/\tau}} \right]_{\text{Text-to-Image Loss}}$$

Learning Transferable Visual Models From Natural Language Supervision

Radford et al. (2021)

Llava: Early-Fusion

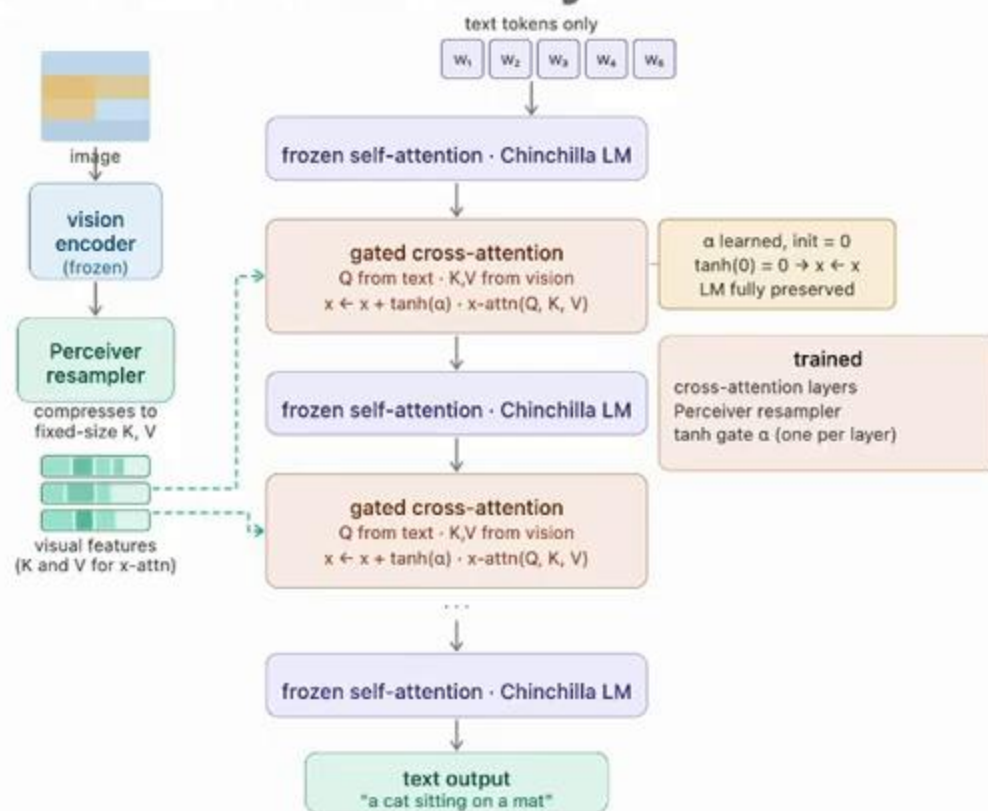
- Prepends image tokens to the text sequence, LLM processes everything in single unified self-attention pass



Visual Instruction Tuning
Liu et al. (2023)

Flamingo: Late-Fusion via Gated Cross-Attention

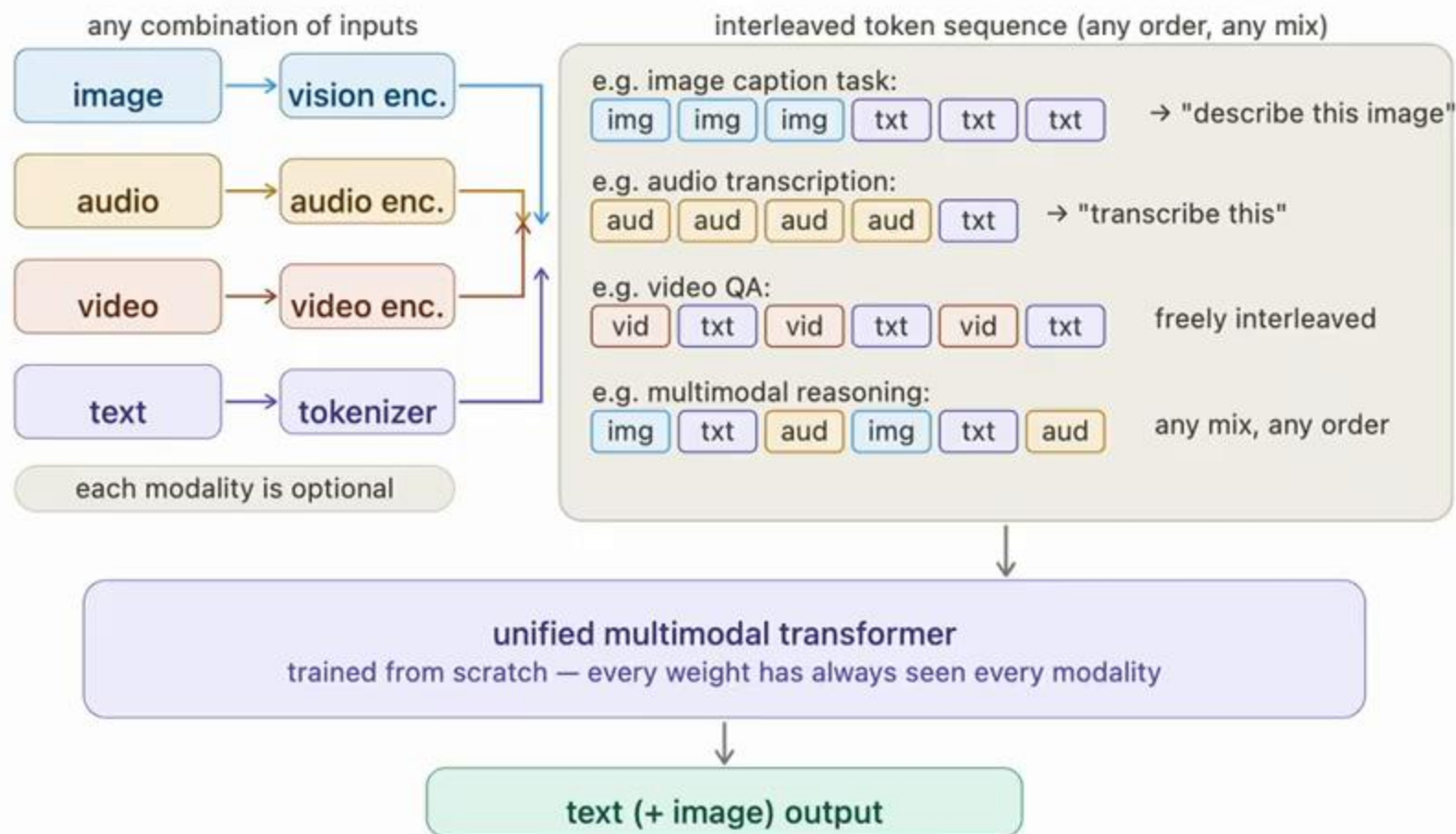
- Vision is injected at every layer via cross-attention, the LM never sees image tokens directly



Flamingo: a Visual Language Model for Few-Shot Learning
Alayrac et al. (2022)

Native Multimodal Models

- Instead of adding vision to an existing LLM, train all modalities jointly from scratch



VLM Taxonomy

early fusion
pre-trained LLM + vision adapter

LLaVA · 2023

CLIP ViT + MLP + LLaMA

PaliGemma 2 · 2024

SigLIP + linear proj. + Gemma 2

InternVL 2.5 · 2024

InternViT-6B + MLP + LLM

DeepSeek-VL2 · Dec 2024

SigLIP + MLP + DeepSeekMoE

Gemma 3 · Mar 2025

SigLIP + linear proj. + Gemma 3

Phi-4-multimodal · Feb 2025

SigLIP-2 + MoE-LoRA + Phi-4

Qwen2.5-VL · 2025

dyn-res ViT + MLP + Qwen2.5

Qwen3-VL · 2025

DeepStack + MRoPE + Qwen3

Mistral Large 3 · 2025

sparse MoE 675B / 41B + vision

late fusion
pre-trained LLM + cross-attention

Flamingo · 2022

gated cross-attn + Chinchilla LM

GPT-4V · 2023

undisclosed · likely cross-attn

LLaMA 3.2 Vision · 2024

cross-attention + LLaMA 3

IDEFICS 2 · 2024

open Flamingo reproduction

NVLM-X · NVIDIA 2024

cross-attn + InternViT-6B

natively multimodal
joint training from scratch

Gemini 1 · 2023

dense transformer · from scratch

Chameleon · 2024

VQ-VAE: images as discrete
tokens in shared vocabulary

GPT-4o · 2024

natively multimodal · undisclosed

Gemini 3 · 2025

sparse MoE · 1M ctx · Deep Think

LLaMA 4 · 2025

MoE · 400B / 17B · 1M ctx

GPT-5 · Aug 2025

native MM · unified routing

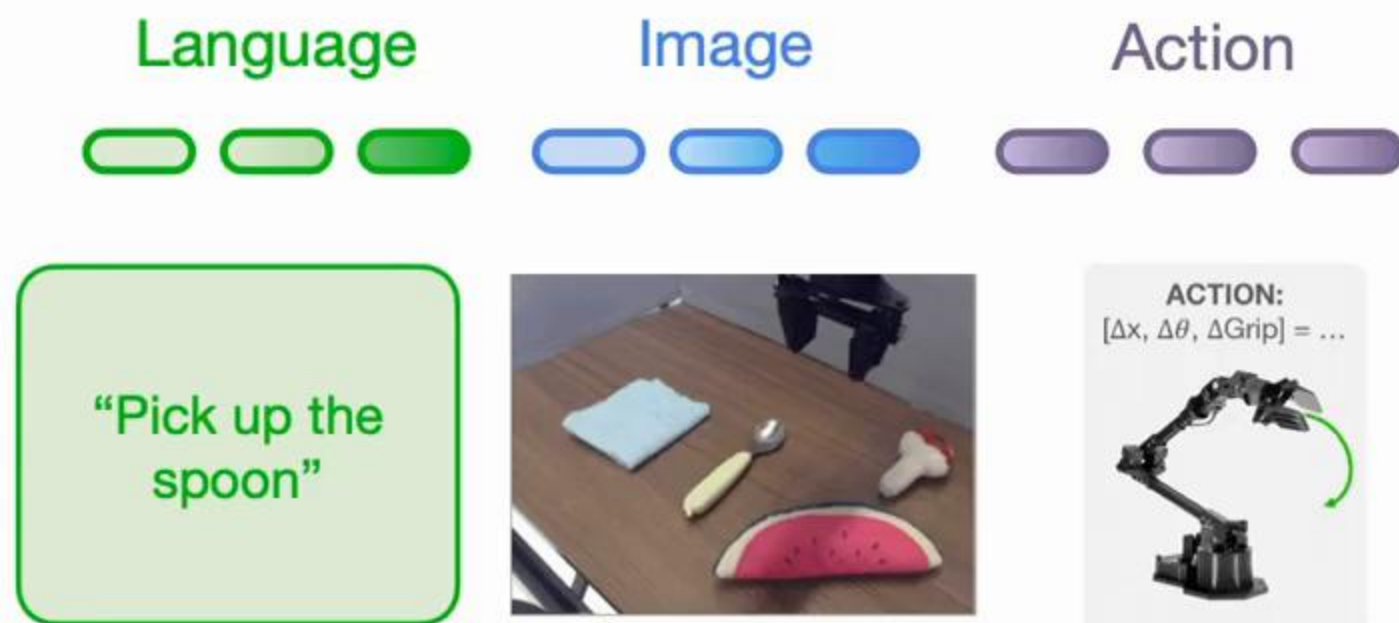
Qwen3.5 · Feb 2026

joint pretraining · text + vision

Transformer → LLMs → VLMs → Native Multimodal models

Can we extend it to robot actions?

Robotics as Multimodal Sequence Modeling

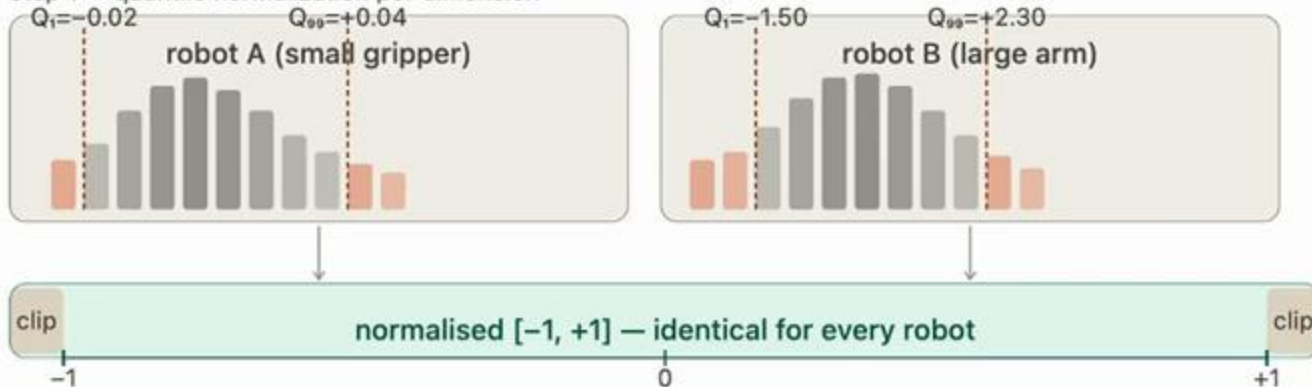


Robot Action Tokenization for VLAs

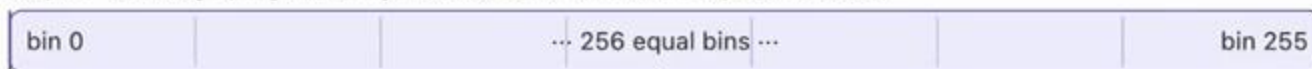
Per-Dimension, Per-Timestep Binning

Quantile bounds prevent outliers from expanding the discretization range and wasting bin resolution

step 1 — quantile normalization per dimension



step 2 — divide $[-1, +1]$ uniformly into 256 bins · bin width = $2/256 \approx 0.0078$



step 3 — N -dim action at timestep $t \rightarrow N$ discrete tokens



step 4 — inject action tokens into pre-trained language model vocabulary · train with next-token prediction

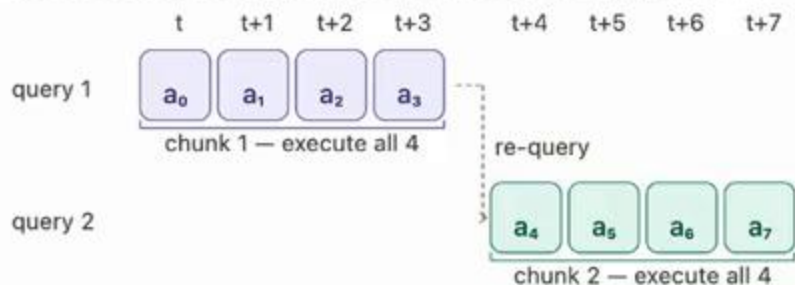


Action Chunking

- Predict k future actions for smoother behavior $\pi(a_{t:t+k} | o_t)$

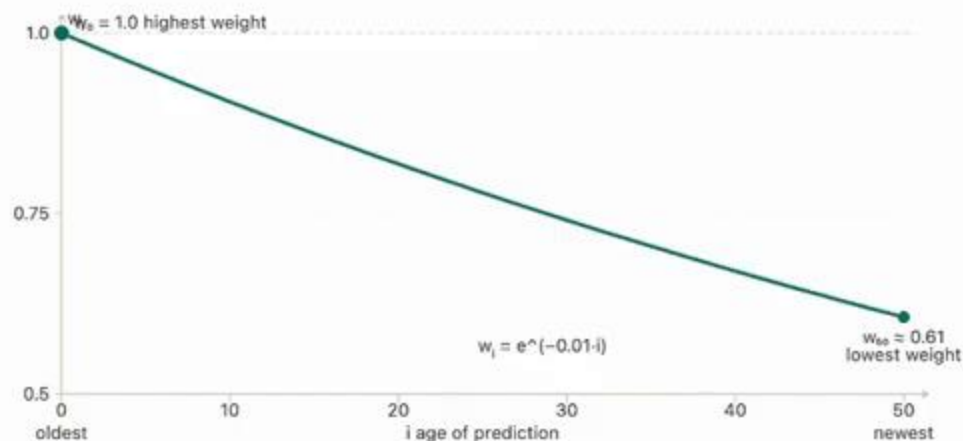
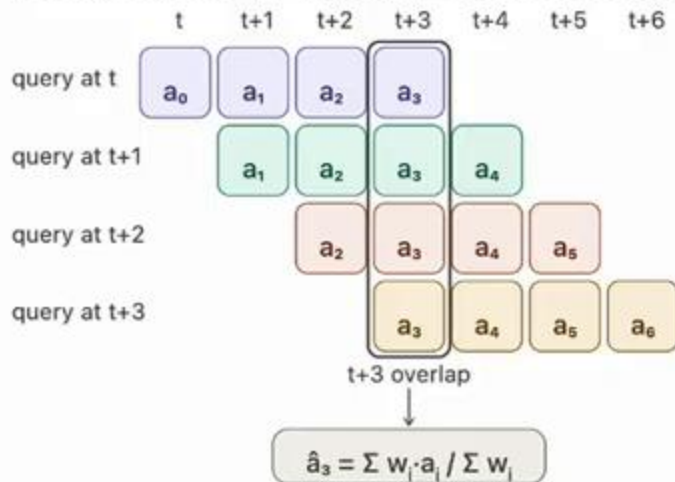
action chunking

predict K actions at once, execute all K before re-querying the model



action chunking + temporal ensemble

re-query every step · $w_i = \exp(-m \cdot i)$ · $i = 0$ is oldest prediction

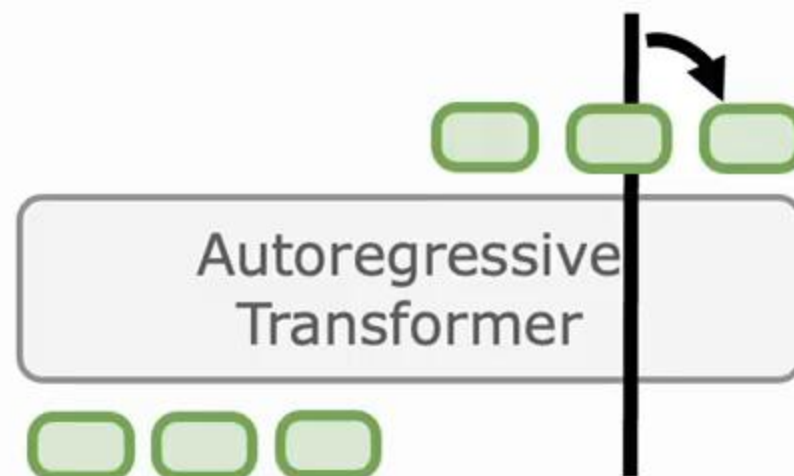
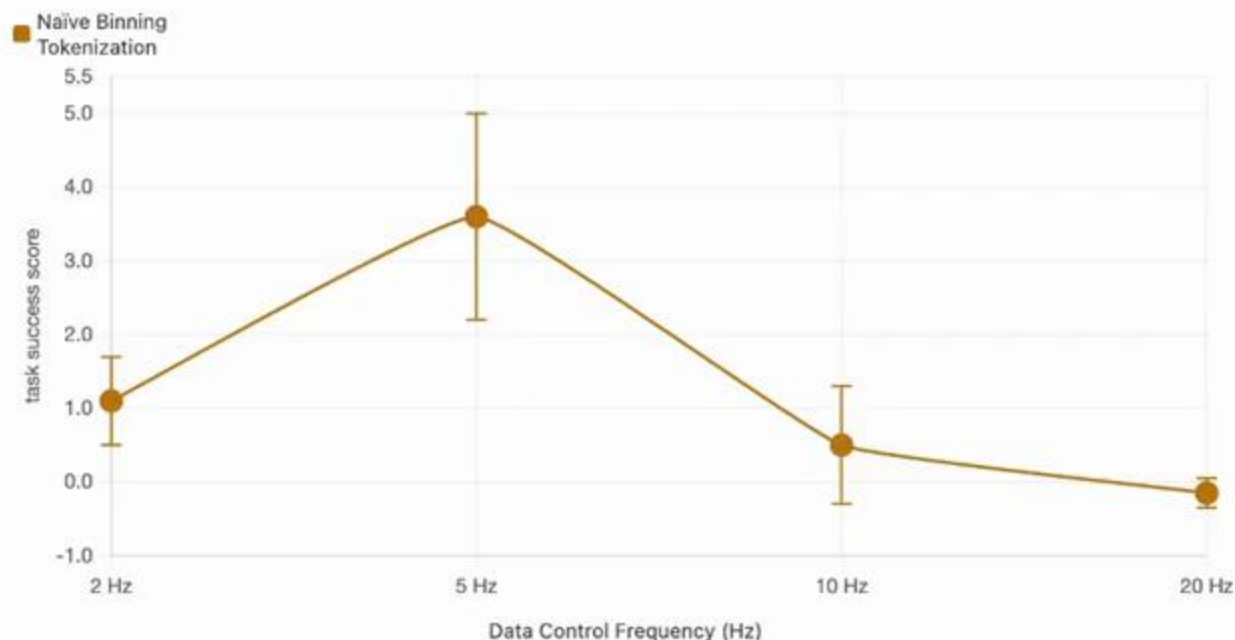


Naïve Action Tokenization

As frequency increases:

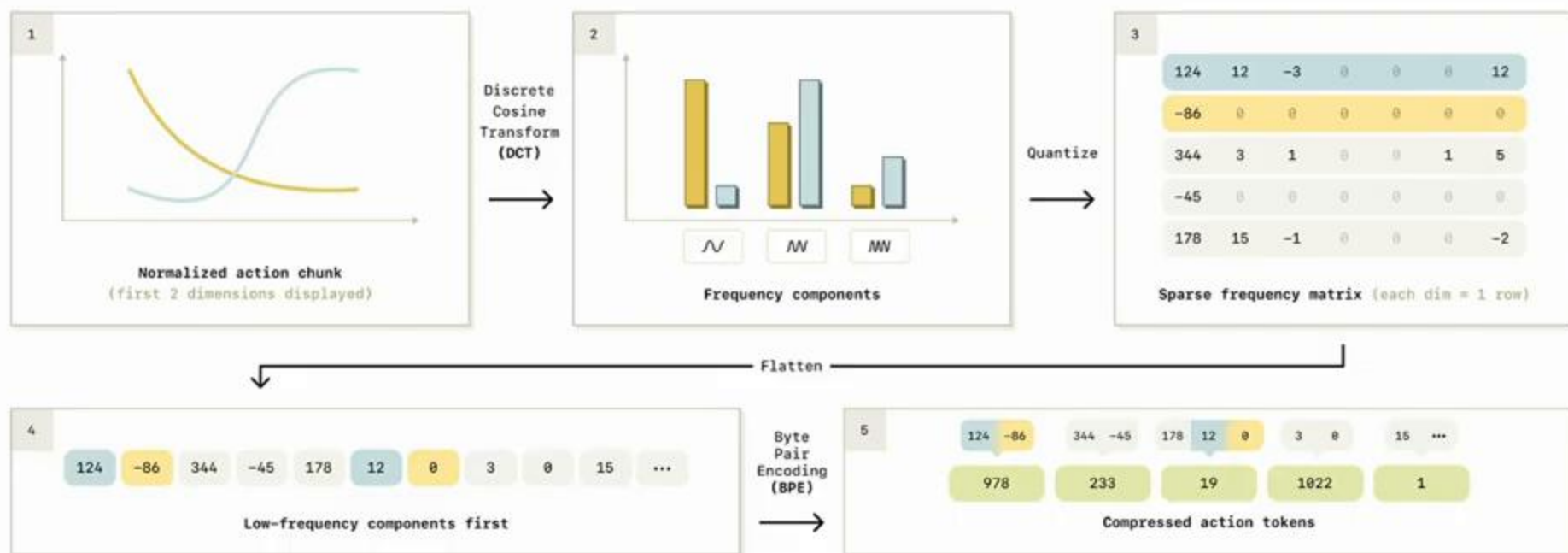
- Information per token decreases
- Correlation between token increases

→ Solution: **compression**



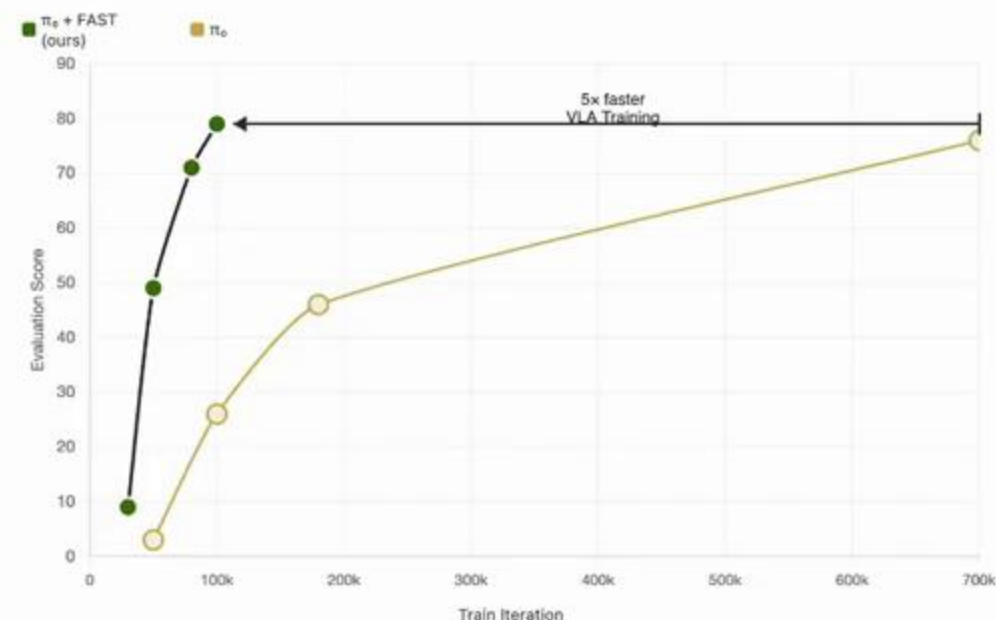
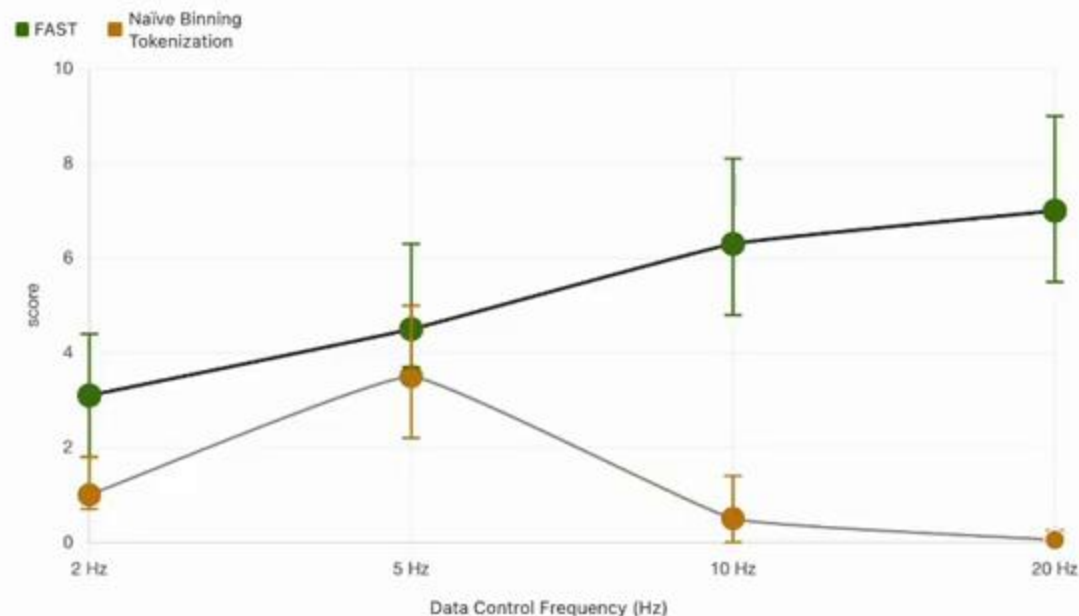
Effective Action Tokenization

- How to compress continuous robot data?
 - Byte-Pair Encoding (BPE) not suited for continuous data
 - VQ learned compression complex to train
- Key idea: **Discrete Cosine Transform (DCT)** $\approx$ JPEG compression



FAST: Efficient Action Tokenizer for VLAs

- Scales to higher frequencies and trains 5x faster



FAST: Efficient Action Tokenization for Vision-Language-Action Models

Pertsch*, Stachowicz*, Ichter, Driess, Nair, Vuong, Mees et al. RSS 2025

Finalist Best Conference Paper Award

FAST: Efficient Action Tokenizer for VLAs

Generality

@ Berkeley, Stanford, UW



Dexterity

@ Physical Intelligence



FAST: Efficient Action Tokenization for Vision-Language-Action Models

Pertsch*, Stachowicz*, Ichter, Driess, Nair, Vuong, Mees et al. RSS 2025

Finalist Best Conference Paper Award

Conclusion

- Robotics is a sequence modeling problem
- Transformers as a scalable architecture → LLMs
- Different architectures for multimodal inputs → VLMs, Native Multimodal models
- Naïve robot tokenization breaks at high frequency control
- Compression-based techniques like FAST popular

Thank you for your attention